Java Topics 1.2

UPDATES IN PROGRESS

JVM memory layout

```
+-------------------------------------------------+
|                    JVM MEMORY                   |
+-------------------------------------------------+
|  Heap                                           |
|  Method Area (Metaspace)                        |
|  Thread Stacks                                  |
|  Native Memory                                  |
+-------------------------------------------------+
```

Each memory contains different pools or region

Heap(Object memory)

Here all objects and arrays live(garbage collected)
String pool is special area inside heap(Java 7+) -> Stores unique literals and prevent duplicates

```
Heap
│
├── String Pool
│   ├── x
│   ├── y
│   └── z
│
└── SomeObject
```

Method area(Metaspace)

```
Metaspace
│
├── Class metadata
├── Method bytecode
├── Static variables
└── Runtime constant pool
        │
        ├── Numeric literals
        ├── String references
```

```
        ├── Method references
        └── Class references
```

Stack(per thread)

```
│
├── Frame (method call)
    ├── Local variables
    ├── References to heap objects
    └── Operand stack
```

System.out.print, why logger over it?

System is a class, out is name of the variable which stores a refrence to PrintStream class, println/print are methods in this class which accept a parameter.
Why logger?
Logger provided benefits like granuility like we can define each message is debug, error, info and warn level. Also they can be disabled at app of package level with some configurations.
Loggers can be created async if we want and can make loggers sensor sensitive data

Interface vs Abstract class

Interfaces

1. Used to define a contract(it specifies what a class must do, but not how it does it).
2. Introduce way for Runtime polymorphism.
3. Before Java 8, they were 100% abstract, but now they can contain:

- default methods: This feature caused diamond problem in java, two interface can have same name default method and a class can implement both and trying to call that common method would cause compiletime issue, to avoid it class needs to provide it's own implementation.
- static methods(can be used as utility)
- private methods (Java 9+ -> Introduce to help static and default methods)

All variables in interfaces are implicitly public, static, final.
Types of interfaces - Functional, Marker, Normal

Abstract Classes

1. Abstract classes are partially implemented parent classes or not implemented at all too. They contain:

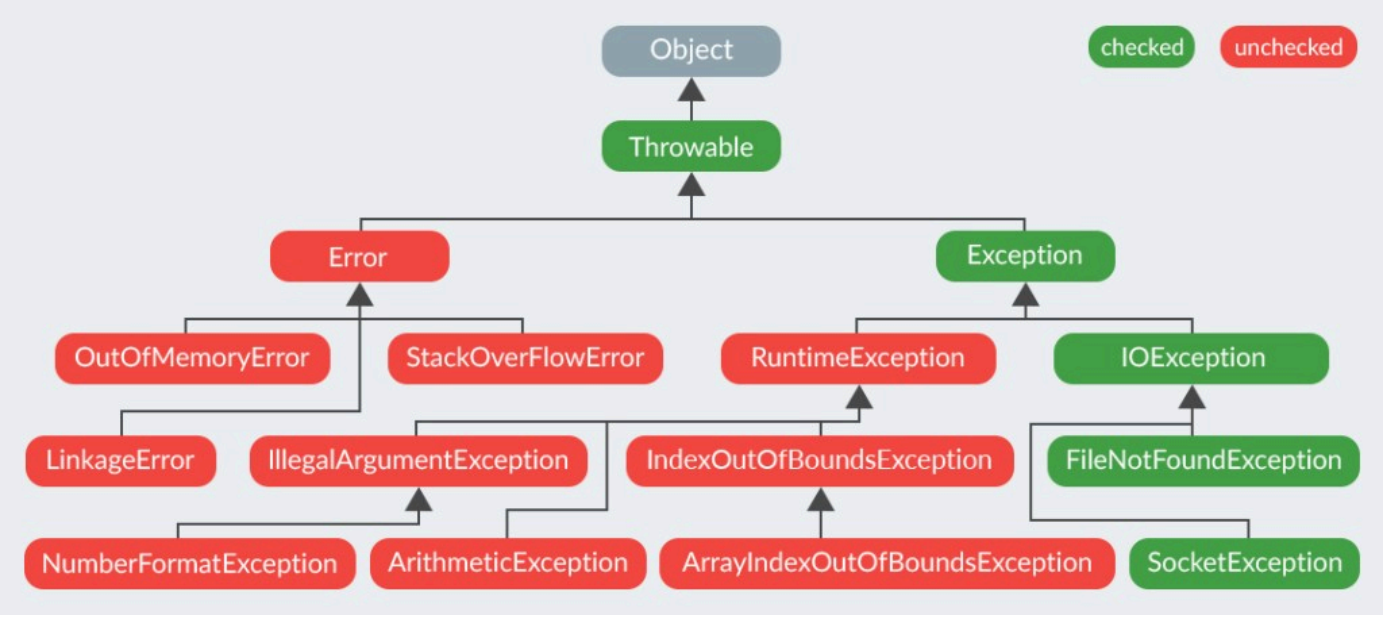
- abstract methods (not necessary)
- concrete methods (not necessary)
- constructors

- non-final fields (state - not necessary)

2. A class need to extend abstract class.
3. Object of abstract class cannot be created cuz they may contain abstract method which java has no idea to find if it exists or not.
4. Why it has a constructor when we can't create it's object? It provides ability to call some methods or set variables as soon as instance of class extending it is created.

-- Prefer composition over inheritence, we can replace them with composition + interface cuz abstract class is tightly binded contract.

Java Exception



Checked Exception

Java verifies checked exceptions at compile-time(this is the reason why compiler gives error if we don't handle exception or throws it)

Unchecked Exception

Java does not verify unchecked exceptions at compile-time. Furthermore, we don’t have to declare unchecked exceptions in a method with the throws keyword.

throws vs throw

throws - It indicates that method may throw one or more checked exceptions
throw - throw is used inside a method or block to explicitly throw an exception.

throw	throws
It is used to explicitly throw an exception.	It is used to declare that a method might throw one or more exceptions.
It is used inside a method or a block of code.	It is used in the method signature.
It can throw both checked and unchecked exceptions.	It is only used for checked exceptions. Unchecked exceptions do not require throws
The method or block throws the exception.	The method's caller is responsible for handling the exception.
Stops the current flow of execution immediately.	It forces the caller to handle the declared exceptions.
throw new ArithmeticException("Error");	public void myMethod() throws IOException {}

Note - the reason checked exception don't work with @Transactional this is the reason since checked exception are always need to be handled or pass on in stack but this behaviour can be configured.

Lambda Expressions

A lambda expression is a short way to implement behavior of an interface with single method(FI) avoids need of creating class implementing it; instance to define it's behaviour.
An interface with only a single abstract method (functional interface) can be implemented using lambdas. default methods implementation can be there in functional interface and private also and also it might implement a marker interface.
Lambdas were introduced in Java 8 and are part of Java's support for functional programming.
A lambda expression is written using the syntax **() -> {}**.

Map (interface)

```
Map (interface)
 |
 +-- HashMap
 |     |
 |     +-- LinkedHashMap
 |
 +-- SortedMap
 |      |
```

```
|       +-- NavigableMap
|             |
|             +-- TreeMap
|
+-- Hashtable
|
+-- ConcurrentHashMap
```

Stores key - value pair where key is unique and value can be duplicate

HashTable

Same as hashmap but synchronised and does not allow null values. Obselete, replaced by concurrenthashmap, used to lock whole array while performing operation.

HashMap

DataStructure with K and V pair
It contains an array of buckets(each index is called bucket) where each bucket can hold multiple entries, before java 8 each bucket contains a linked list. From Java 8+, if bucket entries no. > 8 and no. of buckets equal or more than 64, it is converted to Red black tree. Default capacity is 16.
It will revert back to linked list when number of elements are decreased.

```
if (binCount >= TREEIFY_THRESHOLD - 1) {   // 8 entries
    if (table.length < MIN_TREEIFY_CAPACITY) {
        resize();           // NOT treeify
    } else {
        treeifyBin();       // convert to Red-Black Tree
    }
}
```

HashMap array -> Node<K,V>[] table;
Node Structure ->

```
static class Node<K,V> {
    final int hash;
    final K key;
    V value;
    Node<K,V> next;
}
```

- Resize triggers during 2 conditions, if no of entries > 0.75 * no. of buckets or each bucket contains > 8 entries when array size < 64
- Collision - It happens when two entries produces the same hashes but different equals

- HashMap doesn't contains size variable, while traversing it, counts the size of entries too
- before java 8, time complexity of finding key in bucket was O(n) but after 7+, it's O(log n)
- For null key, java don't call hascode and always store it in Bucket(0)

LinkedHashMap

Basic structure remains same since it's just an extension for HashMap the basic structure remains same, and additional doubly linked list is added.
Whatever key is added, it is also stored is the node, and the next key is added at next node maintaing the sequence in which key is inserted, on successful new addition operation a node is in inserted in doubly linked list. While iterating we can iterate on the doubly linked list directly

It has LRU(Least recently used)/access based mode, while creating linked hashmap (size, loadFactor, mode), if true is passed then if we access/update the element then it will remove the element from it's position in linkedhashmap and will put it at the tail of the linked list. The element at head is least recently used.
eg. Map map = new LinkedHashMap(16,0.75,true);
map.put(1)
map.put(2)
map.put(3)
order -> 1,2,3
map.get(2)
order -> 1,3,2

TreeMap

It does not allow null key, Internally it uses red black tree and sorted by default.
It is basically a redblack tree with one extra varible in node called value.
If the key is object it sorts it using it's natural ordering(i.e. class must implements comparable and define it compare method else we get runtime exception). It's a map but unlike rest it doesn't need hashcode or equals. Since it fundametally works in different way.

HashMap vs ConcurrentHashMap

HashMap

Not thread-safe. Multiple threads modifying it can cause:

1. Data inconsistency
2. Lost updates -> {
   Assume 2 threads simuntaneously try to insert an entry in same bucket and bucket currently is null
   Both creates new node and initialize it in bucket, one can overwrite other and so there will be data loss
   }

3. Infinite loops during rehashing (pre–Java 8) -> {
   if two threads resize at same time, during xopying values they may break the linked list and situation can occur where A->C->A
   }
4. Allows null key and values.
5. No locking mechanism.
6. Read operations may also fail, if one is getting and other insert resizes, it may fail the operation

ConcurrentHashMap

1. Designed for multithreading, doesn't lock the entire map like synchronizedmap/hashtable. Only locks the bucket(index) where write is happening volatile reads for visibility.
2. Java 8 implementation uses,

- CAS (Compare-And-Swap) for first node
- Synchronized blocks only on the specific index
- Multiple threads can read and write concurrently.
- Read operations are lock-free.
- Does not allow null keys/values to avoid ambiguity during concurrency
  All null keys in HashMap go to bucket 0. If multiple threads try to insert null keys concurrently, Which value should survive? Race conditions occur.

Operation-wise summary

Operation	Uses CAS?	Uses bucket lock?
`get()`	❌	❌ (lock-free reads)
`put()`	✅ (if bucket empty)	✅ (if bucket non-empty)
`putIfAbsent()`	✅ (if bucket empty)	✅ (if bucket non-empty)
`computeIfAbsent()`	✅ (empty bucket)	✅ (non-empty bucket)
`replace()`	❌ (no empty insert)	✅
Resize	✅	❌ (no full lock)

Summary
HashMap = fast but unsafe in multi-threading.
ConcurrentHashMap = safe + efficient using fine-grained locking and CAS.

CAS (Compare-And-Swap) (used during update operations)

CAS is a **low-level atomic CPU instruction** used to update a variable **only if it hasn't been changed by another thread**

Mechanism:
CAS takes (address, expectedValue, newValue)
Check if memory location has expected value.
If true → swap with new value atomically.
If false → fail, retry

eg -> count = 10
Thread 1 -> increments count -> (address, 10, 11)
checks if count = expected value ? yes → swap
count = 11
Thread 2 -> increments count -> (address, 10, 11)
checks if count = expected value? no → retries and now expected value is read as 11
During race condition it is made sure that only one thread performs operations at one time

CAS in ConcurrentHashMap Operations
Table Initialization
CAS ensures only one thread initializes the internal array (Node[] table).
First Node Insertion
If bucket empty (null), CAS atomically sets table[i] = new Node(...).
Resizing
CAS coordinates threads during expansion, marking buckets as “moved” to avoid duplicate work.

Race Condition Handling
Scenario: Two threads see table[i] == null simultaneously.
Both attempt CAS.
Winner: CAS succeeds → node placed.
Loser: CAS fails → retries, sees node present, inserts into chain using synchronized block if needed.
Guarantee: Only one thread can transition null → Node.
Result: No corruption, correctness maintained.

Why locks after first node instead of CAS?

CAS works only for a single, independent memory update.
Updating a bucket usually involves multiple dependent steps (traversal + checks + pointer updates).
Those steps must be atomic as a group, which CAS cannot guarantee.
Therefore the bucket is locked.

Special Cases with HashMap/HashSet

Object which is being used as key does not override hashcode - It uses hashcode to decide in which buket element will go, so no hashcode implementation means a default hashcode is generate by object class, means even if 2 equals object are implemented we might still have duplicates in different indexes that too.
Object which is being used as key does not override equals - When hashcode is same hashcollision

occurs and then it uses equls to check elements are same or not, if we don't implements it by default they are not equals even if all values are same and hence duplicate keys will be stored in same bucket

Trees

A DS in which values are represented as nodes and branches are there

String vs StringBuilder vs StringBuffer

String

String is immutable — once created, its value cannot change.
Stored in the String Constant Pool when created using literals.
The JVM reuses the same literal to avoid duplicates.
Creating a string with new String("abc") creates: one object in the heap
Because it’s immutable, any change creates a new String object.

StringBuilder

Mutable — can be modified without creating new objects.
Not thread-safe (no synchronization).
Faster than StringBuffer.
Used in single-threaded environments.

StringBuffer

Mutable and Thread-safe — all methods are synchronized.
Slower due to locking overhead.
Used in multi-threaded environments where safety matters.
Append method in StringBuilder is not synchronised but in StringBuffer is
ref - https://stackoverflow.com/a/26862853

== vs .equals()

== operator
Used to compare references for objects → checks whether both variables point to the same memory location.
For primitives, == compares the actual primitive values.

.equals() method
method in the Object class.
Classes override it (e.g., String, Integer, List, etc.) to compare actual content/value instead of references.

Final vs Finally vs Finalize

final
Used as a keyword in Java.
final variable → value cannot be changed (acts like a constant).
final method → cannot be overridden by subclasses.
final class → cannot be extended.

finally
Used in a try-catch-finally block.
Code inside finally always executes
→ whether an exception occurs or not
→ even if there is a return statement
→ except in rare cases like JVM crash or System.exit().
Used for cleanup operations:
Closing database connections
Closing file streams

finalize()
A method in the Object class.
Called by the Garbage Collector before destroying an object.
Intended for cleanup (like closing resources)
It is deprecated in Java 9+ Because:
It is not guaranteed to run.
Causes performance and reliability issues.

Modern Java uses:
try-with-resources
AutoCloseable

Collection Interface

```
                      Iterable
                          |
                     Collection
                          |
  ____________________________________________
  |                       |                       |
 List                    Set                     Queue
```

Collection interface implements iterable interface and is implemented by 3 major data structure interfaces List, Queue, Dequeue
Core methods

Operation	Meaning
`add(E e)`	Append element
`add(int i, E e)`	Insert at index
`get(int i)`	Read by index
`set(int i, E e)`	Replace element
`remove(int i)`	Remove by index
`remove(Object o)`	Remove by value
`indexOf(o)`	First occurrence
`subList(a,b)`	View over list

List

```
List
|
+-- ArrayList
|
+-- LinkedList
|
+-- Vector
|     |
|     +-- Stack
|
+-- CopyOnWriteArrayList
```

ArrayList

Simply an array, stores duplicate, best for searching via index and adding element at laset or certain index. It is able to resize itself when it is full a new array is created and all elements are copied in other. Once size is increased it is not decreased.
add and get operation is O(1)
rest are O(n)

Vector

Obsolete, basically a synchronised arrayList and is extended by stack. ArrayDequeue is used instead of stack.

Linked list

Contains nodes, each node refrence to next node. Solve the size issue, there is no extra overhead of creating and copying like arraylist
insertion and deletion at head and tail is O(1)

CopyOnWriteArrayList

Create an array copy on each write, reads are lock free and write is free(rarely used). It is *fail-safe* since during write original is not modified but a copy is created. Extremely slow for write operations

Set

```
Set (interface)
 |
 +-- HashSet
 |     |
 |     +-- LinkedHashSet
 |
 +-- SortedSet
       |
       +-- NavigableSet
             |
             +-- TreeSet
```

HashSet

Internally uses HashMap, whatever we add in set are stored as key in the map and the value is some random value like hashmap can store one null.

LinkedHashSet

Internally uses LinkedHashMap, everything same as above, the insertions order is maintained

TreeSet

LinkedHashSet internally uses doubly linked list on top of hashset
TreeSet Implements NavigableSet and SortedSet, backed by a Red-Black Tree.

HashSet/LinkedHashSet/TreeSet

Set Type	Ordering	Speed	Allows Null?
HashSet	No order	Fastest (O(1))	Yes (1 null)
LinkedHashSet	Insertion order	Medium	Yes (1 null)
TreeSet	Sorted order	Slowest (O(log n))	❌ No null

Queue

```
Queue (interface)
 |
```

```
+-- PriorityQueue
|
+-- Deque
      |
      +-- ArrayDeque
      |
      +-- LinkedList
```

Queue → FIFO
Deque → insert/remove from both ends
pop operation - removes top element and provide it
peek operation - returns the value of element in top
push operation - pushes element in structure
LinkedList implements both List and Queue

Priority Queue
Uses Heap datastructure and inserts elements based on their priority, a custom comparator can be defined for it

Serialization/Deserialization

Serialization - It is the process of converting an object's state to a byte stream.
Deserialization - It involves taking a byte stream and converting it back into an object.

Serializable Interface - It is a marker interface found in java.io implementing which makes any class eligible for serialization.
We need to create a serial verion UID when implementing serializable in class, if not java creates a default UID for it.
serialVersionUID - is used to verify compatibility between a serialized object and the class definition used during deserialization. During deserialization UID need to match else **InvalidClassException** error comes.

If a parent is implementing serializable, all it's children automatically becomes serializable too and no straight way to avoid this.

transient keyword - Fields with transient are not serialized.
Using hibernate entity in background serialization/deserialization happens even when we do not implement serialization
eg: -
private transient String password;
transient is not same as @Transient, difference link
https://www.baeldung.com/jpa-transient-ignore-field

Spring Scopes

Scope	Instances Created	Typical Use
singleton	One per Spring container	Services, DAOs
prototype	New instance each time	Stateful beans
request	One per HTTP request	Request-scoped data
session	One per HTTP session	User session data
application	One per application context	ServletContext-level objects
websocket	One per WebSocket session	WebSocket apps

1. ***Singleton (Default Scope)*** Only one instance per Spring container.
   Created at application startup and reused everywhere.
   Most commonly used for stateless services. Not same as singleton pattern
2. ***Prototype***
   A new bean instance is created every time it is requested using getBean().
   Useful for stateful beans or where you explicitly need multiple instances.
3. ***Request (Web Scope)***
   Creates one bean instance per HTTP request.
   Used in Spring MVC/Spring Web apps.
   Bean is created at request start and destroyed when the request ends.
4. ***Session (Web Scope)***
   Creates one bean instance per HTTP session.
   Bean lives as long as the user's session exists.
5. ***Application (Web Scope)***
   One bean instance for the entire ServletContext.
   Similar to singleton, but at the servlet container level (Tomcat/Jetty), not Spring container level.
6. ***WebSocket (Web Scope)***
   One instance per WebSocket session.
   Used for STOMP/WebSocket messaging apps.

To set up a scope use @Scope("name") above a class

Component/Service/Repository/Controller/RestController annotations
@Component
A generic stereotype annotation.
Tells Spring to create a bean of that class during component scanning.
Base annotation for all other stereotypes.

@Service
A specialization of @Component.
Indicates that the class holds business logic, mainly added for readibility but some AOP logic might rely on it so in some cases aop might be affected.

@Repository
A specialization of @Component, but has additional features:
Automatic conversion of SQL exceptions into Spring's DataAccessException hierarchy.
Indicates the class is part of DAO / persistence layer.
Generally no need to use above interfaces

@Controller
Marks a class as a Spring MVC controller.
Used for handling web requests.
Used in Spring MVC for building server-side web applications.
Methods typically return views (like HTML/JSP/Thymeleaf templates)
but with @RestController or @ResponseBody, they return JSON/XML automatically.

@RestController
Combination of @Controller + @ResponseBody.
Every method returns JSON/XML by default (no need for @ResponseBody).
Used for RESTful web services, where you expose APIs (not views).
Typically used in microservices.

Dependency Injection
Dependency Injection (DI) is a design pattern used by Spring where the framework provides the required dependencies to a class instead of the class creating them itself.

This makes code loosely coupled, more testable, easier to maintain, follows Inversion of Control (IoC)
Spring manages all the beans in the ApplicationContext and injects them wherever required.

Types of injection

1. Constructor Injection (Recommended)
   Dependencies are injected through the constructor. Ensures that a class cannot be created without its required dependencies.
   Best for immutability, unit testing, and mandatory dependencies.
2. Setter Injection - Dependencies injected via setter methods, Good for optional dependencies, Bean can be created even without the dependency.
3. @Autowired at field level - Hard to unit test (you must use reflection to set fields). Cannot enforce immutability, Can't use constructor-based dependency checks, causes hidden dependencies.

Why Constructor Injection is preferred?
Makes dependencies final (immutability), Avoids NullPointerExceptions, Enables easier unit testing,

Works better with Spring's AOP and proxies.
Encouraged in modern Spring (especially Spring Boot).

Extra points related to circular dependency behaviour

Constructor injection cannot resolve circular dependencies (A → B → A). Spring fails at startup with BeanCurrentlyInCreationException.

To resolve circular dependency (only works for singleton scope)

1. Field Injection (@Autowired on fields)
   Bean is created first (via default constructor) <- an early refrence.
   Dependencies are injected afterwards via reflection.
   Optional: Can mark @Autowired(required=false) → bean created even if dependency missing (field stays null).
2. Setter Injection
   Similar to field injection: bean created first, then dependencies injected via setters.
   Supports optional dependencies.
   Same circular dependency resolution as field injection.

Spring can resolve cycles using early references.
Example: A depends on B, B depends on A → Spring instantiates A, registers early reference, then wires B with that reference.
Beans are created successfully.
Risk: Class can exist without dependency if required is false, leading to potential runtime NullPointerException.

Circular References
Exist in memory: A → B → A forms a cycle. Not inherently a problem: JVM garbage collector handles cycles fine.
Problem only arises if recursive methods (toString(), equals(), etc.) traverse the cycle endlessly → causes StackOverflowError.
Otherwise, circular references are harmless at runtime.

Bean Lifecycle

1. Instantiation → Bean object created inside Application context.
2. Populate properties (basic DI) → Values from XML/JavaConfig are injected.
3. Aware callbacks → BeanNameAware, BeanFactoryAware, etc.
4. BeanPostProcessor – Before Initialization
   This is where AutowiredAnnotationBeanPostProcessor kicks in.
   It scans the bean for @Autowired, @Value, @Inject annotations and injects those dependencies.
   So technically, annotation-based DI happens here, not in Step 2.

5. Initialization methods → @PostConstruct, afterPropertiesSet(), custom init. Impotant point, post construct can be used to initiate any logic when bean has been constructed.
6. BeanPostProcessor – After Initialization → Proxies, AOP, etc.
7. Bean ready for use.
8. Destruction callbacks.

@Transactional

- Annotation which makes the method gets detected by proxy layer. Whatever DB operation we perform inside this method comes as a part of DB transaction either all commit at once or all rollback at once
- The rollback happens if some exception occures in btw method (only in the case of unchecked exceptions rollback happens. i.e. when the thrown exception is an instance or subclass of RuntimeException. (Error instances also, by default, result in a rollback)).
- A method only comes under transaction if it is **public** and called via another bean, a method calling another method with inside same class doesn't lead to proxy catching it.
- There are proxies between beans, one bean to another bean call goes through proxy that's why above behaviour

Lazy vs Eager in hibernate

1. ***Eager Fetching***
   Means related entities are loaded immediately along with the parent entity.
   Happens at the time of fetching A.
   Example: @OneToMany(fetch = FetchType.EAGER)
   More data is loaded upfront, even if not needed.
   Can cause: Performance issues, Huge joins, Cartesian products, Higher memory usage
   Eager fetching does not always cause 1 + N queries.
   Often Hibernate fires one big join query that fetches A and B together.
2. ***Lazy Fetching (Default for ToMany)***
   Only the parent entity (A) is fetched.
   B is not loaded yet.
   When you access A.getBList(), Hibernate fires additional queries.
   This can cause the classic N+1 query problem:
   Example:
   1 query to fetch all A records
   N queries to fetch B for each A

Fix - Use fetch join to get all data

Links
in depth explaination - https://stackoverflow.com/a/39696775
when to keep n+1 in favour argument - https://stackoverflow.com/questions/97197/what-is-the-n1-selects-problem-in-orm-object-relational-mapping#comment9713570_97253

Docker

It is a tool to package and run software in a container. It is lightweight compared to virtual machine and can be deployed much faster compared to VM.

Using Docker, an app can run consistently across environments where the host kernel supports the same OS kernel family(basically if docker image has linux softwares it won't work with windows kernel).

Docker container - It is a lightweight isolated environment that contains docker image + Writable filesystem layer + shares host OS kernel(basically host cpu, memory, disk, network <- Uses the hose hardware to run the application)
Docker uses the host OS kernel to run containers, while Docker + kernel abstractions (cgroups, namespaces) manage CPU, memory, disk, and network usage.
Docker image - all the softwares needed to run the software are added + application
Normally docker image is read only, writable stuff like logs are written in writable layer

In Azure containers are deployed using Azure Container Instances but normally containers are deployed inside vm and theose vm instances are horizonatlly scaled and all the containers inside vm are managed using kubernetes

Kubernetes

Handles orchestration of containers(basically automates/manages the containers)
Most

JWT Authentication

JWT (JSON Web Token) is a compact, stateless token used for authentication(can be used for authorization also).
It has 3 parts:
Header – contains the signing algorithm (like HS256) base64url encoded
Payload – contains claims such as email, roles, userId which is base64url encoded like header
Signature – ensures the token wasn't tampered with, created using cryptographic method with a secret key(symmetric key)
(Symmetric key example)
Signature is like: HMACSHA256(base64url(header) + base64url(payload) + secret)
JWT structure = `header.payload.signature`
This makes the token tamper-proof, but not private.
(Anyone can decode payload, but they cannot modify it. So passwords should not be stored in it)

Common claim types:

- iss (Issuer): Identifies who issued the token.
- sub (Subject): Represents the user or entity the token is about.

- aud (Audience): Specifies the intended recipient.
- exp (Expiration): Defines when the token expires.
- iat (Issued At): Timestamp when the token was created.
- nbf (Not Before): Specifies when the token becomes valid.

Hashing -> can go from input to output but cannot go other way, JWT is as safe as the secret or private key
links(asymmetric key working depth) -
https://dev.to/kimmaida/signing-and-validating-json-web-tokens-jwt-for-everyone-25fb (article encryption of hash point is incorrect, signing of hash happens not encryption)
https://stackoverflow.com/questions/42644792/verifying-jwt-tokens-rsa?utm_source=chatgpt.com

How is it used for authentication

1. User sends username/password to /login.
2. Backend verifies credentials. If valid it generates a JWT using a secret key.
3. Server returns the JWT, Client stores it (in localStorage or memory).
4. Client sends JWT in every request in Authorization header in form : Bearer + token
5. Server validates signature extracts header + payload, recomputes signature with known secret key. If signature matches → token is valid (basically creates same signature again in symmetric key case)
6. Server extracts user details like roles, userId, email, permissions and stores in Spring Security context

Real life projects follow SSO or oauth. The user is redirected to service provider login page like microsoft authenticator, if login is successfull a token is generated and is given to backend bu UI. backend calls service provider to validate the token and if it is valid it generates a JWT token and refresh token.

Issues with JWT - Once token is ceated it's impossible to invalidate it before it's expire. How to logout then? Blacklisting method is used, in redis layer we store username and token and before each request we can check in cachen once that the specific token is blacklisted or not.

Process vs thread

Process
A process is an independent program in execution.
It has its own memory space, CPU context, and resources.
Communication between processes is heavy and slow.
Example: Running Chrome, VS Code, and your Java app — each is a separate process.

Thread
A thread is a lightweight unit of execution inside a process.

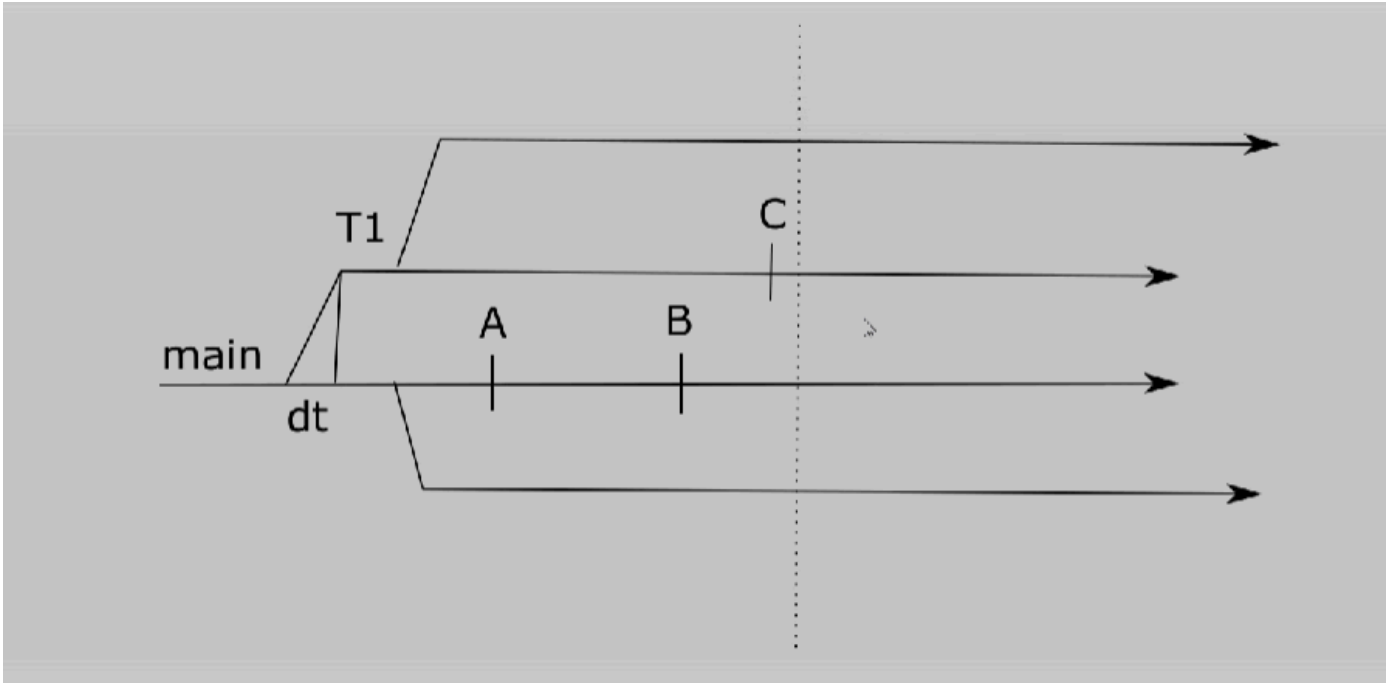
Multiple threads share the same memory of the process.
Communication between threads is very fast.
Used for concurrency — handling tasks in parallel.

🧠 Key Differences

Feature	Process	Thread
Memory	Separate memory	Shared memory
Creation cost	High	Low
Communication	Slow (IPC)	Fast
Failure impact	One process failing doesn't affect others	One bad thread can crash whole process
Use case	Running apps	Tasks within an app

Thread

A set of instructions which execute in defined order and parallel to other threads
-> Starting a new thread takes time shown by dt in diagram.
-> Due to this all parallel instructions do not run at same time
-> In multithreaded structure it's impossible to guess which instruction will be executed at exact time parallely, C can run before A or between A and B or after B.




```
//To make a class Thread
class Test extends Thread{

    @Override
```

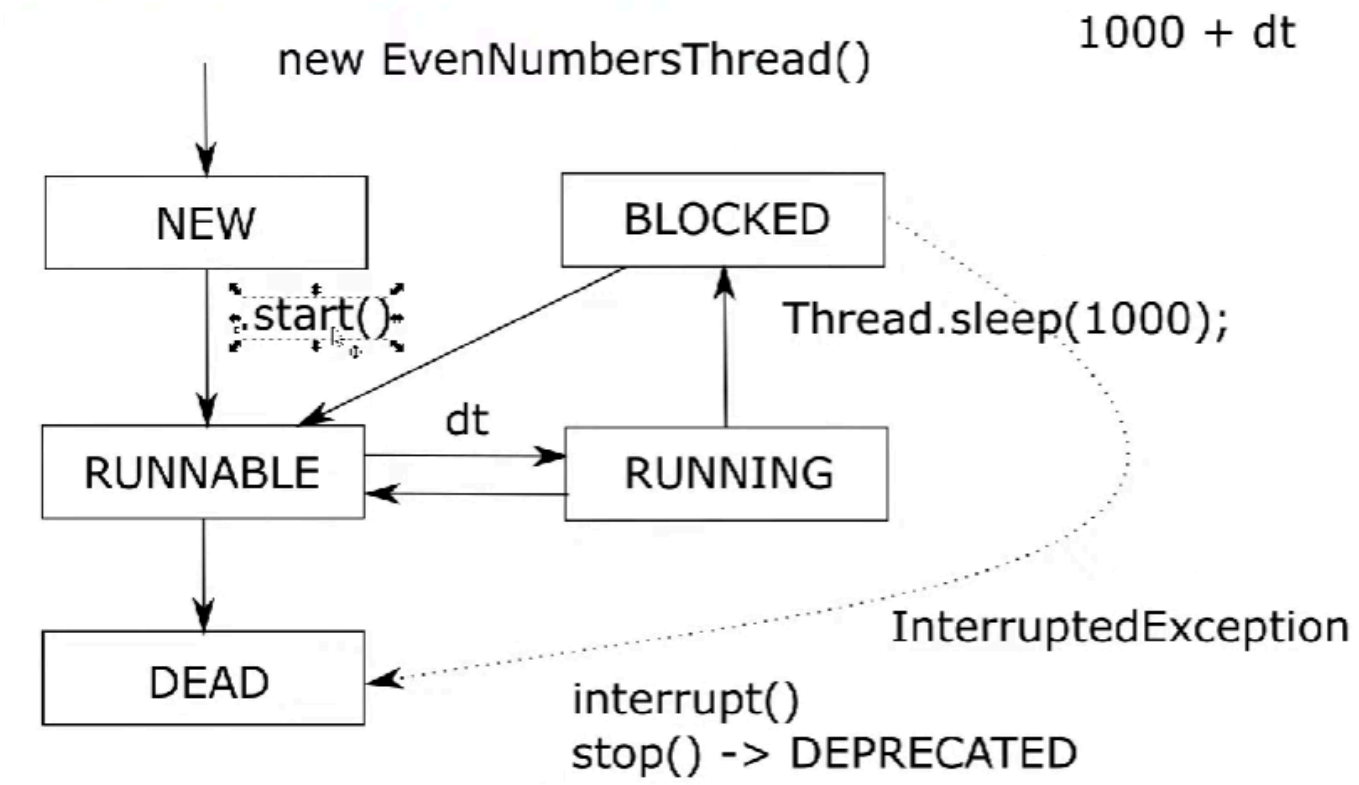
```
    public void run(){ //main method of thread
        for(int i = 0; i< 100; i+2){
            sout(i);
        }
    }
}

class Main{
    main(){
        Test thread = new Test();
        thread.start(); //this is one actually starting the thread
        sout("end");
    }
}
```

-> Above case 'end' keyword can be printed anywhere but most likely situation is end is printed at start due to 'dt'. The thread had not started at the point but all outputs are possible.

Lifecycle of a Thread

Step 1: During creation of thread it goes in **new** state, just it's object goes in heap pool
Step 2: When we call *start()* method it goes in **Runnable** state but maybe or may not be running(depends on your CPU core is free or not).
Step 3: Whenever CPU is free it can go in **Run** state, takes *dt* time for this.
(
Extra Steps -
Step 3.5: From **Run** state, thread can go in **Blocked** state when we call *Thread.sleep()* and from there in **Runnable** state.
Step 3.6: If *interrupt()* or *stop()* is called on a blocked thread by any other thread, it puts thread in dead state and throws **InterruptedException** a checked exception.
)
Step 4: If thread job is finished, it goes in **Dead** state. We cannot start thread in dead state, if we try to call *start()* we get **IllegalStateException**, however we can create new instance and start it.



Note - To get thread name Thread.currentThread().getName().
-> Thread cannot be controlled between **Runnable** and **Running** state.

Imlementing interface is preffered over directly creating thread.

Runnable Interface

-> This interface has only one method, *run()*. It's a functional Interface that does nothing.
-> It just allow more flexibility over extending class and provide decoupling
-> Just extending Thread class you are not required to override run but interface forces you to implement run.

```
//To make a class Thread using Runnable
class Test implements Runnable{
    //override run
    @Override
    public void run(){ //main method of task
        for(int i = 0; i< 100; i+2){
            sout(i);
        }
    }
}

class Main{
    main(){
```

```
        Runnable task = new Test();
        //class implementing runnable is passed in constructor of Thread class
        Thread thread = new Thread(task);
        thread.start(); //this is one actually starting the thread
        sout("end");
    }
}
```

methods

sleep(time in ms) -> blocks thread for passed time (rare in real world)
join() ->

- Assume we have 2 database and need data from both in end, if we have 2 threads. Possible ways to get both data.
- 1. Run 1st thread wait for it to end then another, 3+2 = 5sec.
- 2. Get data after Max(3, 2). Join does this but not exactly this way, it stops the current thread till other thread which we used join on completes.

```
class Test{
    public static void main(String[] args){
        //since runnable is a functional interface able to use lambdas instead of
creating a class to define it's functionality
        Thread a = new Thread(() -> {
            try {
                Thread.sleep(3000);
                System.out.println("a");
            } catch (InterruptedException e) {
            }
        });

        Thread b = new Thread(() -> {
            try {
                Thread.sleep(3000);
                System.out.println("b");
            } catch (InterruptedException e) {
            }
        });
        a.start();
        b.start();
        //Since in blocking state it can thorw Interrupted exception so handle it
        try{    //makes main thread in waiting state till a finishes
```

```
            a.join();   //in parameter we can define waiting time also
        } catch(InterruptedException e){
        }
        sout("processing");
    }
}
```

setPriority() -> Puts the priority of thread from 1-10. Not really a way to control threads. It is just an advice. Default priority 5 and never use in real project. Controlling threads should be done with synchronization
yield() vs join() -> While join() stops current thread till other thread completes, puts the thread in waiting state while in yield thread moves to runnable from running state and surrenders CPU to equal priority thread but this will happen is not guranteed unlike join.

synchronized keyword

-> Makes the block can be only entered by a single thread at a time and others won't be able to enter till one thread exits. It is unfair, any thread can break line.
example with producer and consumer
Step 1: Create Producer

```
class Main{
class Producer extends Thread{
    @Override
    public void run(){
        Random r = new Random();
        while(true){
            if(Main.bucket.size() <=100)
                Main.bucket.add(r.nextInt());
        }
    }
}
```

Step 2: Create consumer

```
class Consumer extends Thread{
    @Override
    public void run(){
        Random r = new Random();
        while(true){
            if(!Main.bucket.isEmpty()){
                System.out.println(Main.bucket.remove(0));
            }
        }
```

```
    }
}
```

Step 3: Create 2 objects for each and start thread

```
class Main{
    public static List<Integer> bucket= new ArrayList<>();

    public static void main(String[] args) {
        Producer producer = new Producer();
        Producer producerA = new Producer();
        Consumer consumer = new Consumer();
        Consumer consumerA = new Consumer();
        producer.start();
        consumer.start();
        producerA.start();
        consumerA.start();
    }
}
```

-> Above logic will stop after sometime, since ArrayList is not synchronized and IF array list has only one element and C1 remove it then C2 try to remove from empty array it breaks the logic. To fix use synchronized keyword

Step 4: Introduce synchronized with arraylist as monitor

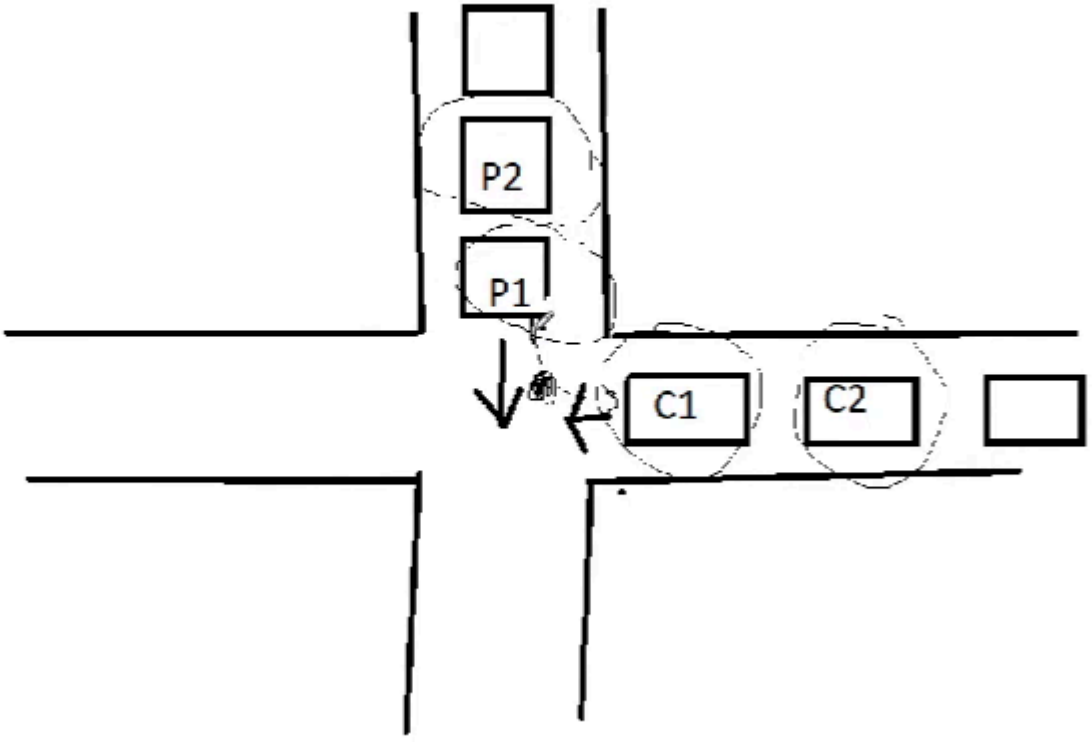
```
class Producer extends Thread{
    @Override
    public void run(){
        Random r = new Random();
        while(true){
            synchronized (Main.bucket){ //monitor is needed in parameter, any
object can be used.
                if(Main.bucket.size() <=100)
                    Main.bucket.add(r.nextInt());
            }
        }
    }
}
class Consumer extends Thread{
    @Override
    public void run(){
        Random r = new Random();
        while(true){
            synchronized (Main.bucket){
                if(!Main.bucket.isEmpty()){
```

```
                    System.out.println(Main.bucket.remove(0));
                }
            }
        }
    }
}
```

-> here both synchronised blocks are synchronised among themselves since there is a single monitor which is managing both.

Monitor(synchronized(monitor))

-> Monitor is passed as parameter to synchronized, it can be any object; that object will manage the synchronized block. Why main bucket is passed in both instead of this or 2 different class or any other thing?
-> If there were 2 different objects managing lock producer and consumer can access the arraylist at sametime and one might delete element while other might add, condition can happen where there could be data corruption because removal means shift of all elements but at the same time some element addition could be happening



synchronized on a method is same as creating a synchronized block with **this** as monitor

notify() and wait()

-> These methods are part of Object class and are used with Monitors means always used inside synchronised block and if used outside it gives runtime error.

-> wait() releases the lock from block and go in blocked state, so another thread can enter inside it and to make thread go again in runnable state we use notify().
-> Notify generally tells the next thread to release but we are not sure next thread will be our blocked thread so we can use notifyAll() method
Expanding above example, assume the array is empty/full and consumer tries to remove from it/producer tries to add. So it will check if condition again and again and will unnecessary use CPU to avoid this performance issue we will use wait, same in producer. wait() releases the lock from synchronised so another thread can enter.

```
class Producer extends Thread{
    @Override
    public void run(){
        Random r = new Random();
        while(true){
            synchronized (Main.bucket){ //monitor is needed in parameter, any
object can be used.
                if(Main.bucket.size() <=100){
                    Main.bucket.add(r.nextInt());
                    Main.bucket.notifyAll();
                }
                else
                    Main.bucket.wait();
            }
        }
    }
}
class Consumer extends Thread{
    @Override
    public void run(){
        Random r = new Random();
        while(true){
            synchronized (Main.bucket){
                if(!Main.bucket.isEmpty()){
                    System.out.println(Main.bucket.remove(0));
                    Main.bucket.notifyAll();
                }else
                    Main.bucket.wait();
            }
        }
    }
}
```

-> Assume we are in Consumer 1 we find bucket empty then we make thread goes in blocked state and it release the lock, next if C2 turn it finds bucket empty and it goes in blocked state too, instead of C2

directly producer thread turn can come too, after adding item it tells other to go in runnable state and loops go on. Other possibilities can also happen.

volatile keyword

-> Normally Threads can work on seperate CPU cores and each core has it's own cache memory, using code when thread modifies any common resource, the resource don't get updated in main memory instantly but updated in cache.
-> If other thread required the same variable value, it will see the old value not updated value.
-> To avoid this situation we use volatile keywords on variable, in this way the value is directly updated in memory during update and it is read from main memory too.

Daemon Thread

- It is a low-priority background thread they run in background to support user threads(normal threads).
- To make any thread daemon thread use thread.setDaemon(true);
- Example Garbage collector and finalizer

Use Cases

- Garbage Collection: The Garbage Collector (GC) in Java runs as a daemon thread.
  Background Monitoring: Daemon threads can monitor the state of application components, resources, or connections.
- Logging and Auditing Services: Daemon threads can be used to log background activities continuously.
- Cleanup Operations: Daemon threads may periodically clear temporary files, release unused resources, or perform cache cleanup.
  link -https://www.geeksforgeeks.org/java/daemon-thread-java/

Garbage collector lifecycle

https://medium.com/@AlexanderObregon/the-mechanics-behind-javas-memory-management-and-object-lifecycles-2895f1bd62ab

Executor Service

- Provides a **higher-level API** for managing threads compared to manually creating Thread objects. Provides the benefit of reusing the Threads instead of manking new threads and creating it again. They are Lazy by default and only start creating initial threads when we start the task.
- Main configuration of thread pool
  1. executor.setCorePoolSize(2) - These are minimum threads which are created when tasks are given.
  2. executor.setQueueCapacity(100) - After 2 threads are created and tasks keep on lining.

3. executor.setMaxPoolSize(5) - When the queue treshold is broken it adds other thread till 5 threads. These threads are removed when the queue finally becomes empty.
4. Additional Case - When threads max limit is reached and our queue is still going 103+ the additional tasks are rejected with *TaskRejectedException*

@Async

It is an annotation provided in Spring boot and is used above methods, when we call the method using a bean which is annotated a parallel thread is run and executes the task. By default it uses *SimpleAsyncTaskExecutor*(not same as executor service,, not using pool). It just automates Thread creation, use it to run method and in end destroys it for us.

- To use an executor service with this annotation we need to configure an executor service by implementing AsyncConfigurer

```
import org.springframework.context.annotation.Configuration;
import org.springframework.scheduling.annotation.AsyncConfigurer;
import org.springframework.scheduling.concurrent.ThreadPoolTaskExecutor;

import java.util.concurrent.Executor;

@Configuration
public class AsyncConfig implements AsyncConfigurer {

    @Override
    public Executor getAsyncExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(2);
        executor.setMaxPoolSize(5);
        executor.setQueueCapacity(100);
        executor.setThreadNamePrefix("AsyncExecutor-");
        executor.initialize();
        return executor;
    }
}
```

- Other way - Create a bean and just provide bean name in annotation eg : @Async("bean name")

```
@Bean(name = "fastExecutor")
    public Executor fastExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(5);
        executor.setMaxPoolSize(10);
        executor.setThreadNamePrefix("FastExec-");
        executor.initialize();
```

```
        return executor;
    }
```

Why use Executors (ThreadPoolExecutor) for creation of threads instead of creating Threads manually

Creating threads manually is expensive. Each new Thread() allocates: memory for stack, OS resources, scheduling overhead. If we create many threads, the JVM can crash or slow down and Thread creation is not scalable.

Thread Pools reuse threads, ThreadPoolExecutor maintains a fixed number of threads. When a task comes in it creates a thread(lazy nature) if not there and when it is completed it keeps it in Runnable state so it can be reused
If all threads are busy → task waits in queue. This avoids creating new threads again and again.

Callable vs Runnable

Both are functional interface and used with Threads
Runnable
Represents a task that does not return a result, Cannot throw checked exceptions and has only one method *void run()*

Callable
Represents a task that returns a result, can throw checked exception since the original call method throws Exception. Interface has one method *V call()* V can be any value

🧠 Key Differences

Feature	Runnable	Callable
Return value	No	Yes
Exceptions	Cannot throw checked exceptions	Can
Method	run()	call()
Supports Future?	No	Yes

Implementing a runnable thread

```
class MyTask implements Runnable {

    @Override
    public void run() {
```

```
        System.out.println("Thread running: " + Thread.currentThread().getName());
    }
}

public class Main {
    public static void main(String[] args) {

        Runnable task = new MyTask();     // create Runnable
        Thread thread = new Thread(task); // pass Runnable to Thread
        thread.start();                   // start new thread
        //via lambda
        Runnable task = () -> {
                System.out.println("Thread running: " +
Thread.currentThread().getName());
        };
        new Thread(task).start();
    }
}
```

run vs start vs call

run() → method in Runnable. If you call it directly, it runs in the current thread.
call() → method in Callable. You don't call it directly; the executor calls it when running the task.
start() → method in Thread. It creates a new thread and internally calls run().

Why thread class has run() method?

The Thread class implements Runnable due to which it overrides run. When we call *start()* method, the JVM creates a new thread and calls that thread's run() method. If we call *run()* directly it is just invoking a normal method in the current thread.

Future vs CompletableFuture

Future class represents the result of an asynchronous task. It is blocking in nature if we want to apply some operations on the tasks we need to wait for the task to complete. Provides us method like *get()*, *isDone()*, *get()* blocks current thread and wait for the task to complete(just like *join()*) and return result.

CompletableFuture (Java 8) - Basically an extension of future, unlike future it is non-blocking in nature. Can Combine results of two of more async task using methods like *thenApply()* and if we want to apply any operation on the result we won't need to wait for the result.
When we use it's methods like *supplyAsync(task, executor pool)* and don't provide the executorpool name then by default it uses forkjoinpool.

Deadlock
A deadlock occurs when two or more threads are permanently blocked, each waiting for a resource that

the other thread is holding.
As a result, none of the threads can continue, and the program freezes.

How Deadlock Happens (Simple Example)
Thread A has Lock 1, and needs Lock 2
Thread B has Lock 2, and needs Lock 1

Both threads:
hold one lock
wait for another
neither releases the lock, Deadlock happens

```
synchronized(lock1) {
    synchronized(lock2) { }
}

synchronized(lock2) {
    synchronized(lock1) { }
}
```

Why Deadlocks Are Dangerous

- Threads get stuck forever
- CPU usage may spike
- Application may hang
- Hard to debug in production

Race Condition
A race condition happens when two or more threads access and modify shared data at the same time, and the final result depends on which thread runs first.
Because thread scheduling is unpredictable, the output becomes inconsistent and unreliable.
Example:
Two threads updating the same bank balance at the same time → incorrect final balance.
How to prevent Race Conditions in Java

You prevent them by ensuring only one thread updates shared data at a time:

- synchronized keyword
- Ensures only one thread can execute a block/method.
- Locks (ReentrantLock)
- More advanced locking with features like tryLock(), fairness, timeout.
- Atomic variables (AtomicInteger, AtomicReference)
- Use lock-free thread-safe operations like compareAndSet.

- Thread-safe Collections
- ConcurrentHashMap, CopyOnWriteArrayList, etc.

synchronized block vs method
Only one thread can access a synchronized block or method at a time.
The synchronized keyword is used for this purpose.
A block that starts with the synchronized keyword is called a synchronized block, and if we write synchronized on a method, it becomes a synchronized method.
Only one thread can execute them at a time.

Locking Types

Lock implementations

Reentrant Lock

System design

List of all the components nedded to design a system, any component can be utilized in any way in real life, like multiple masters and all, no universal design all depends on requirement. All components are like pieces of blocks which can be used to create anything.
Note -> UI is in client browser(CSR) the frontend is not on our server.

1. DNS - Just the name of actual website, linked with IPAddress of our frontend
2. DB - NoSql or Sql, Unstructured data? nosql else relational
3. Scaling - Vertical? In Same instance add more CPU power(Very Low scale), Horizontal? increase web server instances (Lagre scale)
4. Load Balancer - Distribute traffic eqully so one instance doesn't overload
5. Databse replication - 1 master for write, slaves for read. Usually apps are read heavy but if write heavy this design can be changed to need
6. CDN - Cache the data like images, videos, etc in CDN network

Steps to Scale system

Step 1. Everything is on a single web server db, backend, etc.

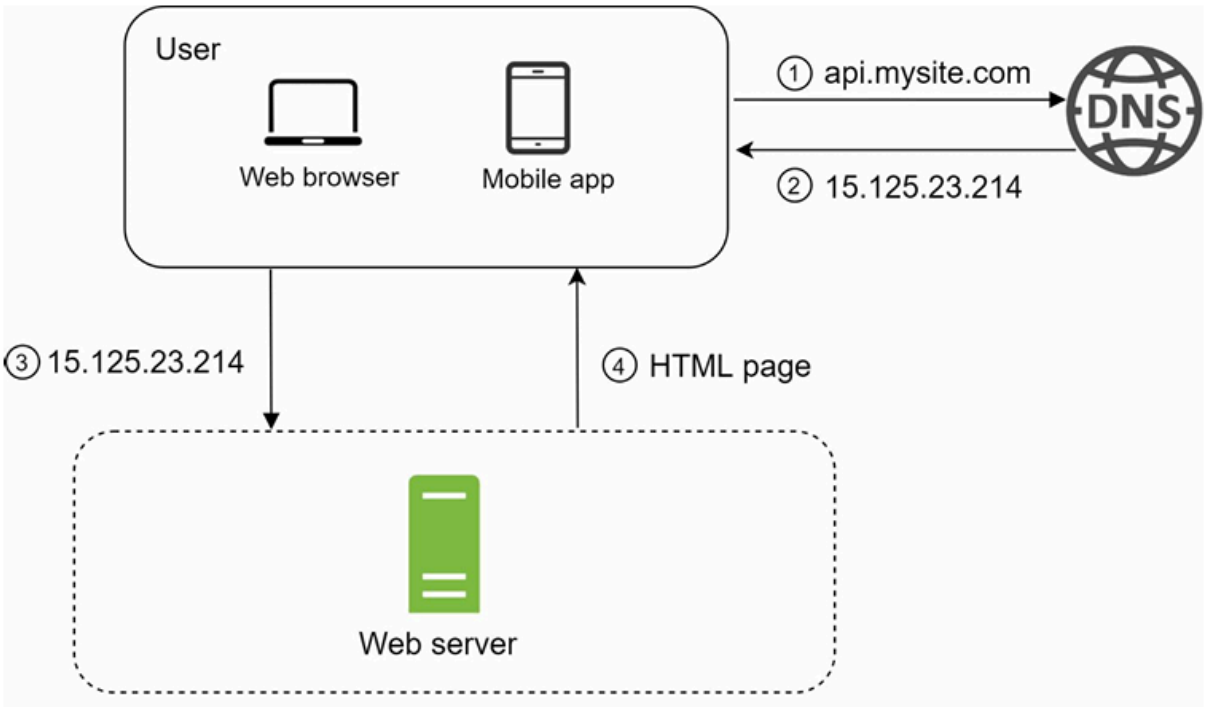
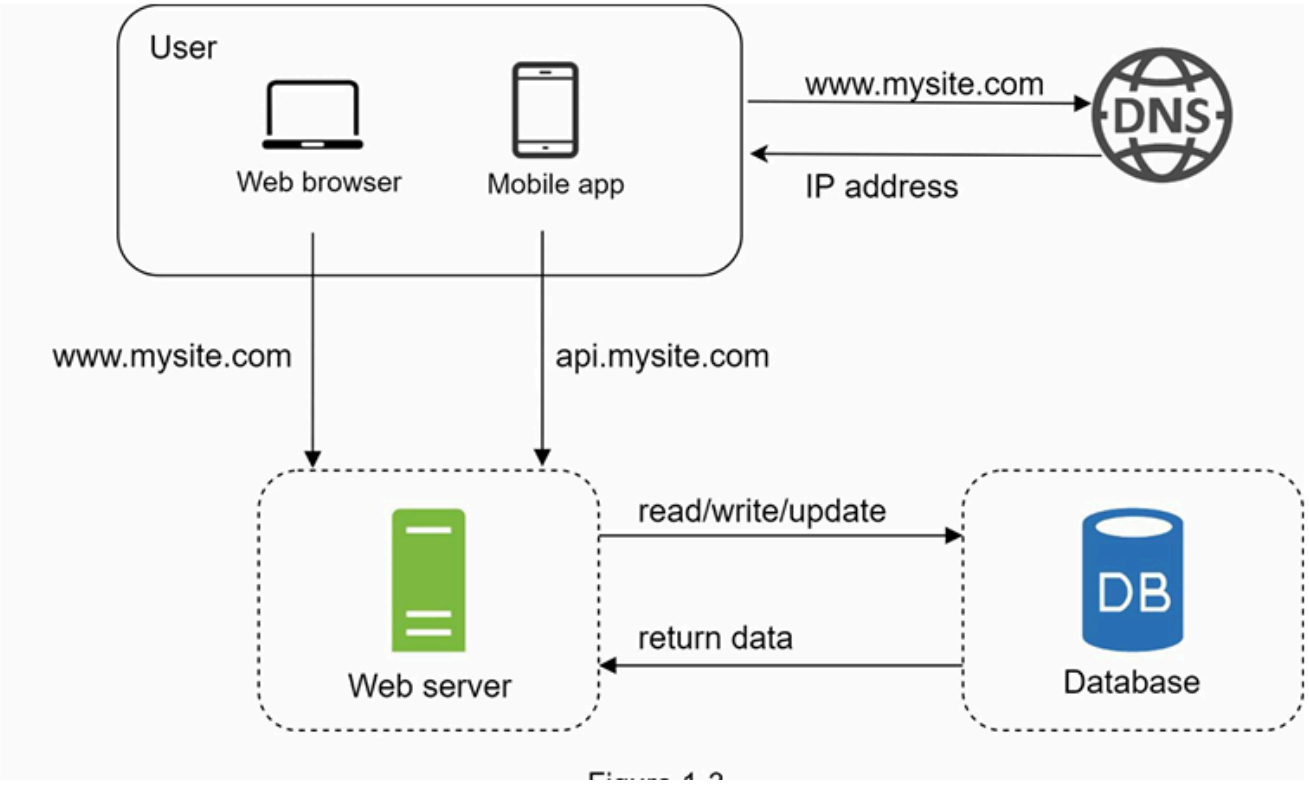
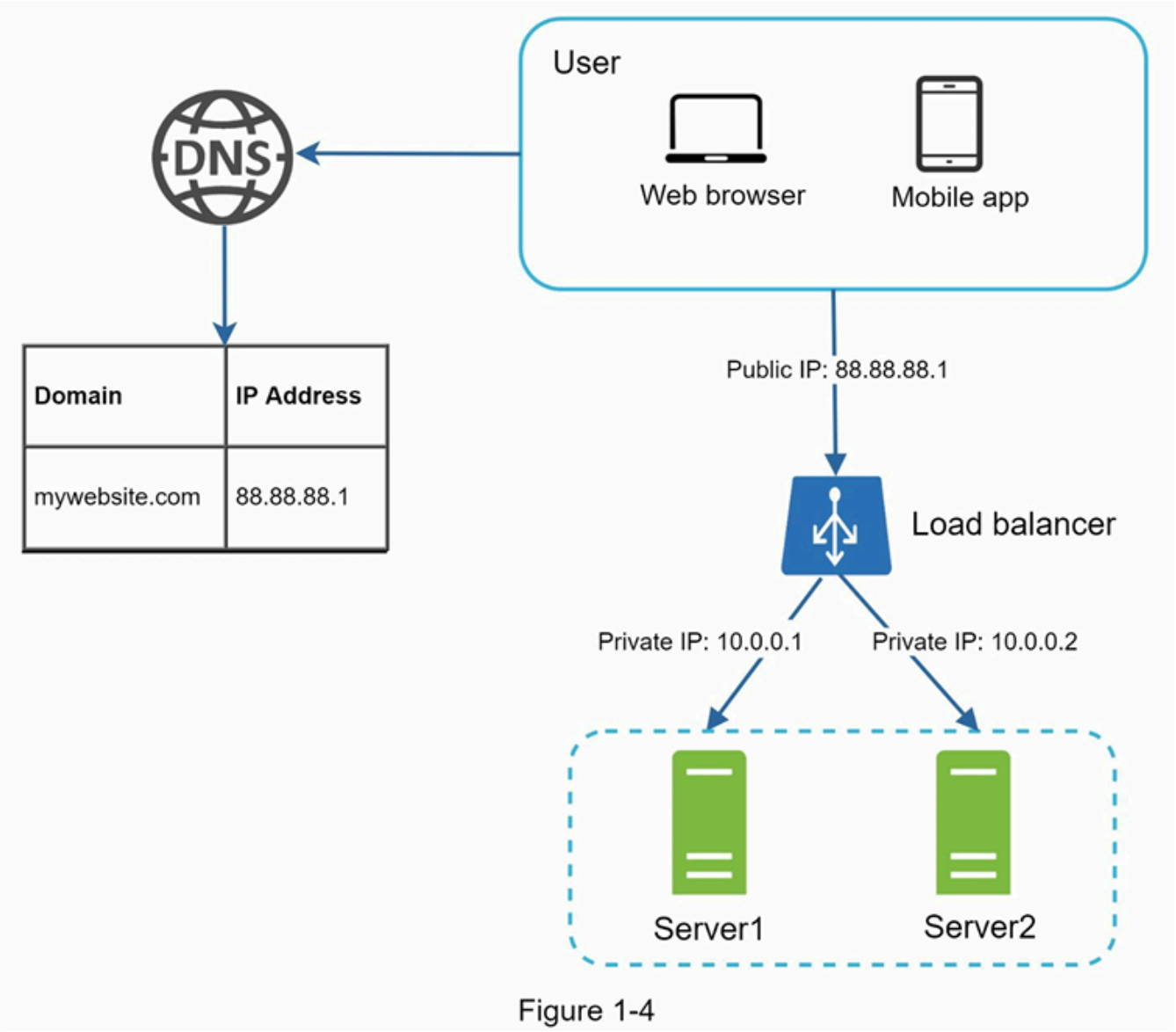


Figure 1-2

Step 2. Seperate DB from our server - This way we can scale both independently



Step 3. Start horizontal scaling and add load balancer to handle traffic divertion just automate instance addition/removal. Why more instance? If one server goes online so whole app don't die



Step 4. Database replication - Usually one Master used for write only and slaves for read. Slaves can be spread throughout different regions for low read latency. Whatever is written in master is replicated to slave but due to latency can take time.

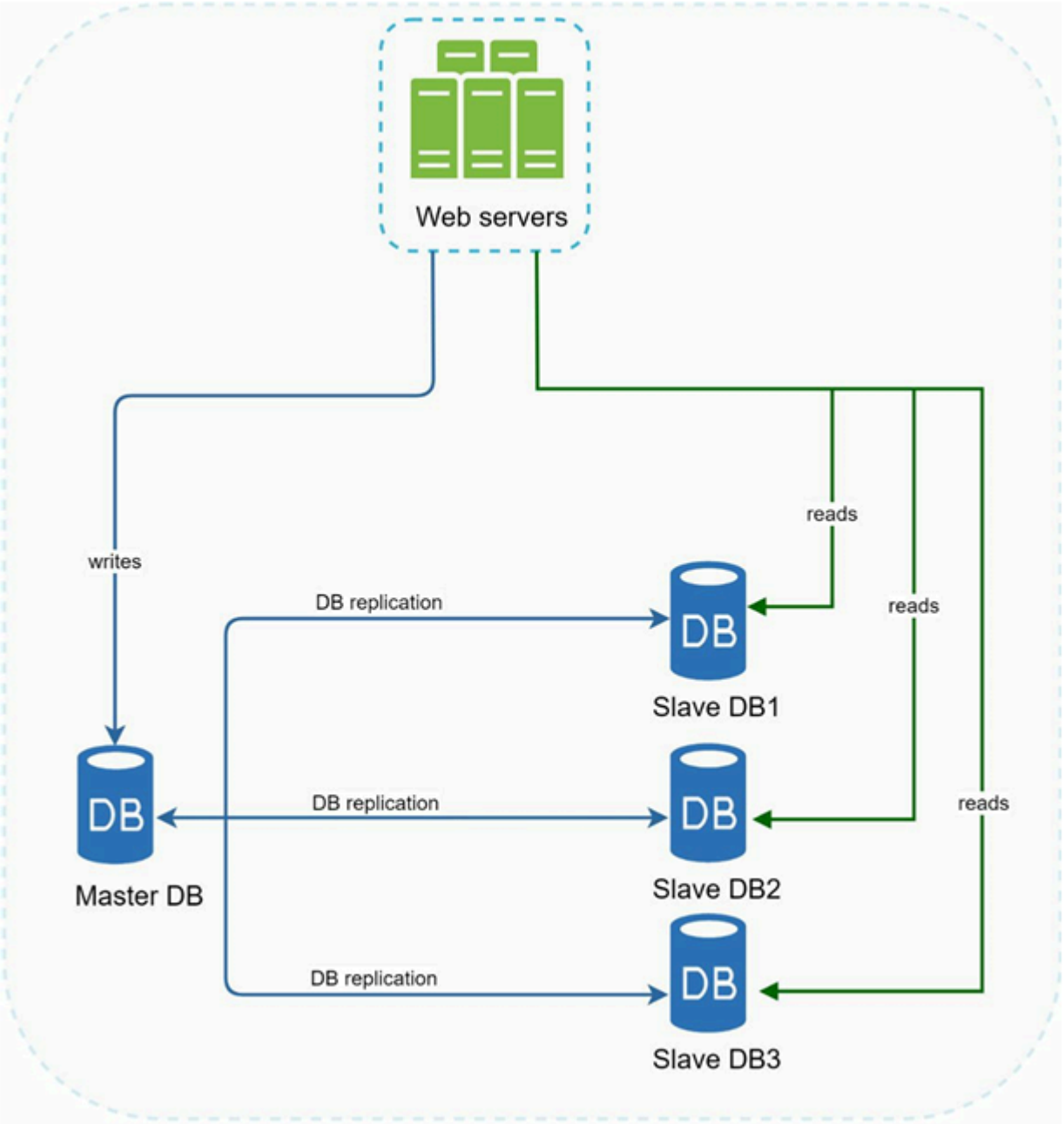
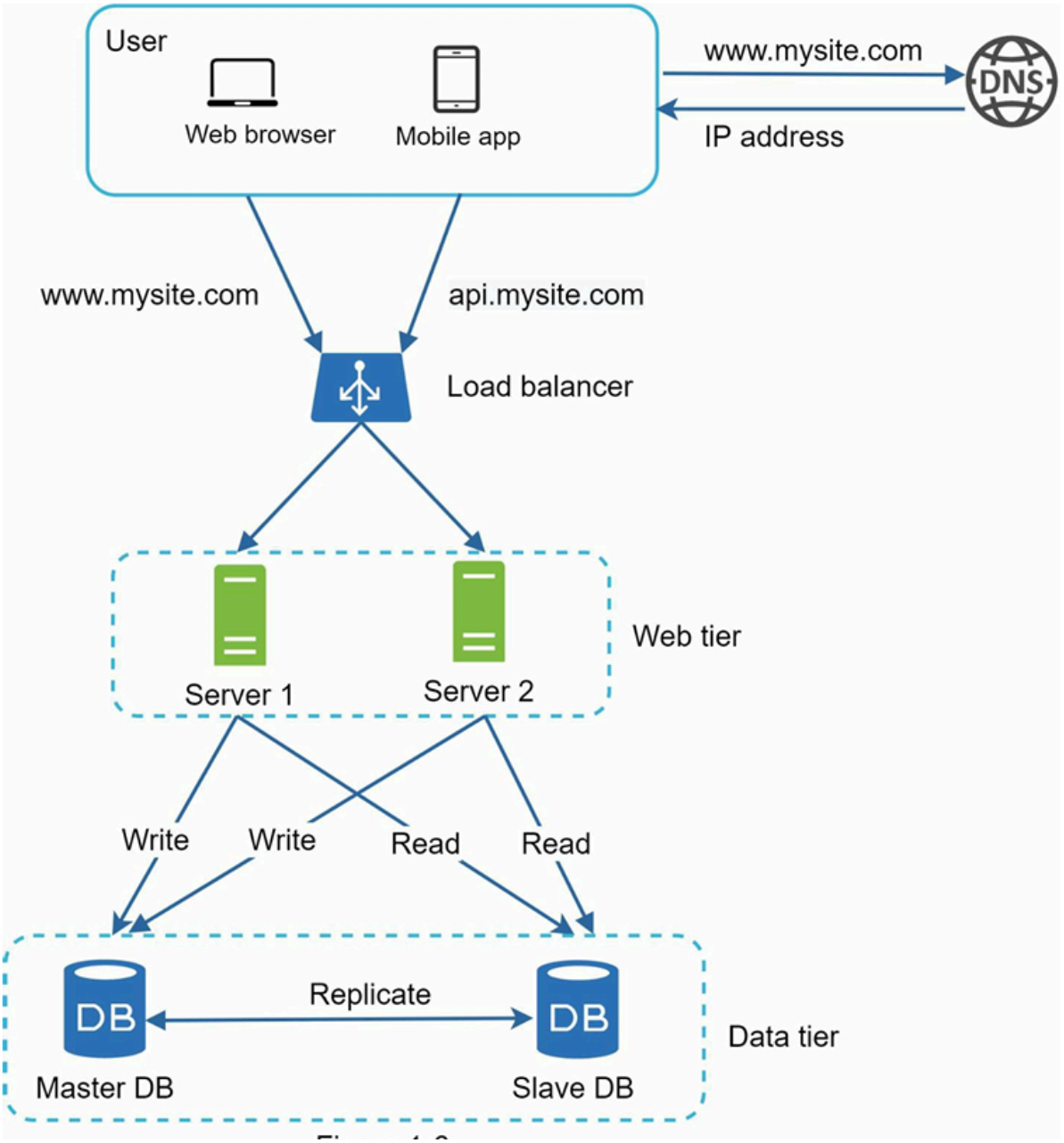


Figure 1-5

Some cases -

• If only one slave database is available and it goes offline, read operations will be directed to the master database till new slave comes.

• If the master database goes offline, a slave database will be promoted to be the new master. Promoting a new master is more complicated as the data in a slave database might not be up to date. The missing data needs to be updated by running data recovery scripts. Although some other replication methods like multi-masters and circular replication could help, those setups are more complicated.



Step 5. Introduce cache layer CDN(Static images, large files, videos), Redis(Api responses, session). Each requests first check for response in cache if not available it gets from DB then add in cache.
Note - It is SPOF - single point of failure, if theire is just one cache server if it fails whole app fails. Why? Cache store important data like session and repeated requests, absense of cache overload db with all hits and due to which whole system might slow down. And threads going to cache also gives timeout..

Cache follows LRU policy for eviction, other policies(LFU, FIFO)

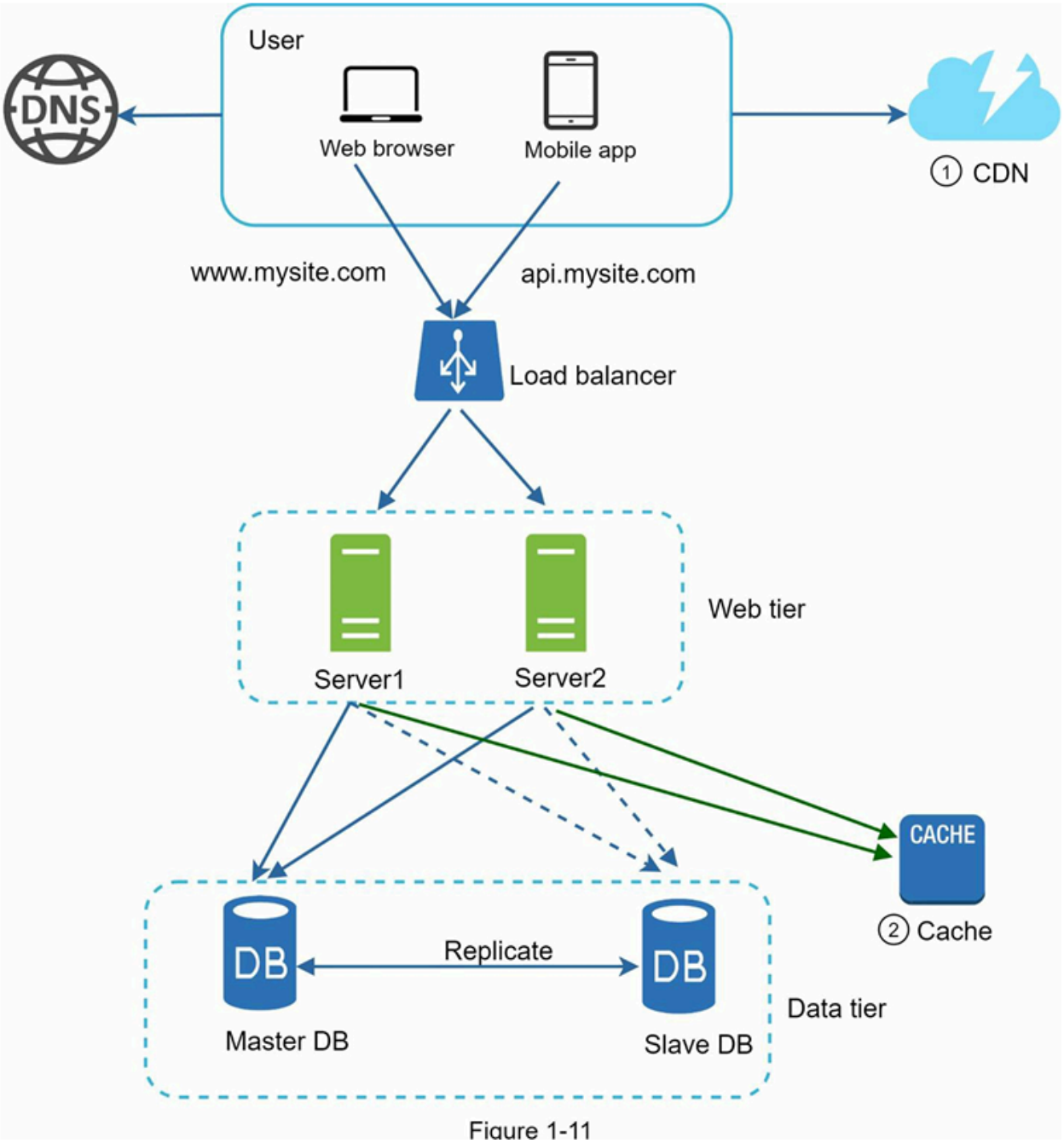


Figure 1-11

Common LLD patterns

Immutable class

A final class with no setters but getters and a constructor, all fields private and final also

```
public class Employee{
    private final int id;
    private final String name;
    Employee(int id, String name){
        this.id = id;
        this.name = name;
    }
```

```
    //getters
}
```

Creational Patterns

Singleton

Always give same instance of the object globally.
Lazy instantiation
Step 1. Make constructor private, so creating object class outside is impossible

```
public class Singleton{
    private Singleton(){
    }
}
```

Step 2. Make class create it's instance

```
public class Singleton{
    private Singleton(){
    }
    public static Singleton getSingleton(){
        return new Singleton;
    }
}
```

Step 3: Above can give multiple instance, track the instance now using static variable

```
public class Singleton{
    private static Singleton obj = null
    private Singleton(){
    }
    public static Singleton getSingleton(){
        if(obj == null){
            obj = new Singleton();
        }
        return obj;
    }
}
```

Lazy one don't work in multi threaded architecture - due to both can try to initialize at same time
Step 4: make method synchronized (cause performance issues)

```
public class Singleton{
    private static Singleton obj = null
    private Singleton(){
```

```
    }
    public static synchronized Singleton getSingleton(){
        if(obj == null){
            obj = new Singleton();
        }
        return obj;
    }
}
```

Step 5: Make object creation only synchronised

```
public class Singleton{
    private static Singleton obj = null
    private Singleton(){
    }
    public static Singleton getSingleton(){
        if(obj == null){
            synchronized(Singleton.class){ //why parameter? u need something that
can be synchronized and static means nothing exists
                if(obj == null)
                    obj = new Singleton();
            }
        }
        return obj;
    }
}
```

Eager instantiation

```
public class Singleton {
    public static Singleton instance = new Singleton();
}
```

Better way - Singleton with nested class, during instance creation class is loaded and classloader itself is synchronised and hence class is loaded only once so nested class will loaded once. Inner classes are not loaded until called when getSingleton is called it is loaded in synchronized way. Faster due to JVM handled

```
public class Singleton{
    private Singleton(){
    }

    public static Singleton getSingleton(){
        return SingletonInstance.instance;
    }
```

```
    private static final class SingletonInstance{
        private static final Singleton instance = new Singleton();
    }
}
```

eg class : Runtime
-> Spring Singleton is closer to registory pattern

Factory Method

For loose copuling, gives responsibility to create objects to a seperate class instead of creating manually in code everywhere and hide it's implementation.
Step 1: Define interface

```
interface Vehicle{
    void start();
}
```

Step 2: Create actual objects

```
class Car implements Vehicle{
    @Override
    void start(){
        System.out.print("square");
    }
}
```

Step 3: Create Factory which provides instances on objects

```
class CarFactory{
    public static Vehicle getinstance(){
        return new Car();
    }
}
```

eg in java - LocalDateTime, Path

Abstract Factory

Factory of factories - expanding above functionality

```
interface Vehicle{
    void start();
}

class Bike implements Vehicle{
    @Override
```

```
    void start(){
        System.out.print("start");
    }
}

class Car implements Vehicle{
    @Override
    void start(){
        System.out.print("start");
    }
}

class BikeFactory{
    public static Vehicle getinstance(){
        return new Bike();
    }
}
class CarFactory{
    public static Vehicle getinstance(){
        return new Bike();
    }
}
```

Step 1: Remove static functionality from factories and create an abstract/interface factory which will give other factories object and other factories will implement it too and interface will return other factories.

```
interface VehicleFactory{
    Vehicle getInstance();
    static VehicleFactory getFactory(Type type){
        if(type == Type.CAR)
            return new CarFactory();
        else if(type == Type.Bike)
            return new BikeFactory();
    }

    enum Type{
        CAR, BIKE;
    }
}
class BikeFactory implements VehicleFactory{
    public Vehicle getinstance(){
        return new Bike();
    }
}
```

```
class CarFactory implements VehicleFactory{
    public Vehicle getinstance(){
        return new Bike();
    }
}
```

eg -> In hibernate EntityManagerFactor, SessionFactory

Builder

Builder solves the problem of creating too many constructors for creation of objects with variables. With builder we can create object with any sequence or number of variables.

```
class Person{
    private int id;
    private String name;
    private String color;

    static class Builder{
        private Person person = new Person();
        public Builder id(int id){
            person.id = id;
            return this;
        }
        public Builder name(String name){
            person.name = name;
            return this;
        }
        public Builder id(String color){
            person.color = color;
            return this;
        }

        public Person build(){
            return person;
        }
    }
}
```

eg: Locale class, userDetails in spring security